

# Computer Vision Assignment 1: Automated Image Analysis

Arnav Kapoor

September 2025

## Project Overview

This project presents a comprehensive pipeline for automated image analysis using a custom dataset (`genai_image_dataset`) containing thousands of images across multiple semantic classes (e.g., airplane, cat, dog, tree, etc.). The folder is organized for reproducible research and modular experimentation. All code is written in Python 3.12, using only open-source libraries: `numpy`, `matplotlib`, `scipy`, and `requests`. No proprietary or platform-specific dependencies are used, ensuring portability and ease of use.

## Folder Structure

- `code/`: Utility scripts for dataset handling and setup.
- `src/`: Main scripts for Q1–Q4, each implementing a distinct image processing task.
- `genai_image_dataset/`: The core dataset, organized by class subfolders, each containing hundreds of images in various formats (JPG, PNG, GIF, etc.).
- `images/plots/`: Output images and plots, organized by question and class for easy review.
- `report/`: LaTeX report and generated PDF summarizing the project.

## Dataset Details

The `genai_image_dataset` folder is a rich resource for computer vision experiments. Each class subfolder (e.g., `airplane`, `cat`, `dog`, `tree`, etc.) contains a diverse set of images, enabling robust analysis and generalization. For each task, 200 images are randomly sampled from all classes to ensure both diversity and computational efficiency. The dataset structure supports scalable experimentation and can be easily extended with new classes or images.

## Scripts and Processing Pipeline

### Q1: Color Channel Analysis and Grayscale Conversion

- **Theory:** Digital color images are typically represented in the RGB color space, where each pixel has three values corresponding to Red, Green, and Blue intensities. Channel separation allows analysis of color distribution and image content.
- **Formulas:** Grayscale conversion uses the luminance formula:

$$I_{gray} = 0.299R + 0.587G + 0.114B$$

This formula weights the channels according to human visual sensitivity.

- **Implementation:** Each sampled image is split into R, G, B channels and visualized. Histograms of pixel intensities are plotted for each channel, providing insight into color distribution and image characteristics.

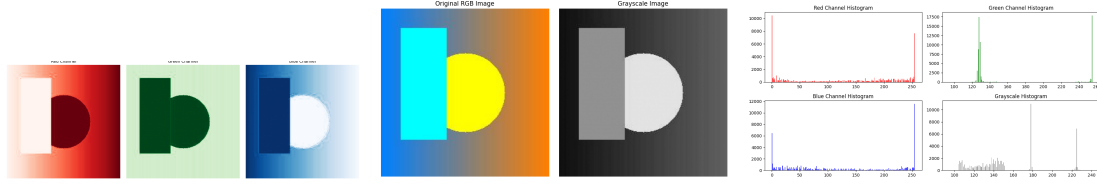


Figure 1: Q1 Example: Color channels, grayscale, and histogram.

## Q2: Gaussian Noise Addition

- **Theory:** Gaussian noise models random variations in pixel values, simulating sensor noise or transmission errors. It is characterized by a normal distribution with mean  $\mu$  and standard deviation  $\sigma$ .
- **Formulas:** For each pixel  $I$ , noisy value  $I'$  is:

$$I' = I + N, \quad N \sim \mathcal{N}(0, \sigma^2)$$

- **Implementation:** Gaussian noise is added to each image. Noisy channels and their histograms are visualized, demonstrating the impact of noise on image statistics and downstream processing.

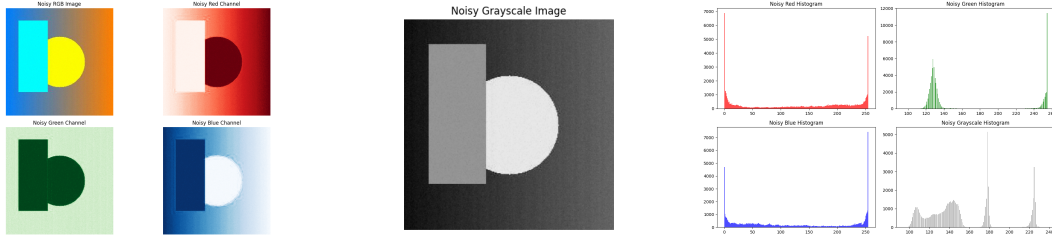


Figure 2: Q2 Example: Noisy channels, grayscale, and histogram.

## Q3: Edge Detection (Sobel, Laplacian)

- **Theory:** Edge detection highlights boundaries and transitions in images. The Sobel operator computes gradients, while the Laplacian detects regions of rapid intensity change.
- **Formulas:** Sobel filters:

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Gradient magnitude:

$$M = \sqrt{G_x^2 + G_y^2}$$

Laplacian filter:

$$L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

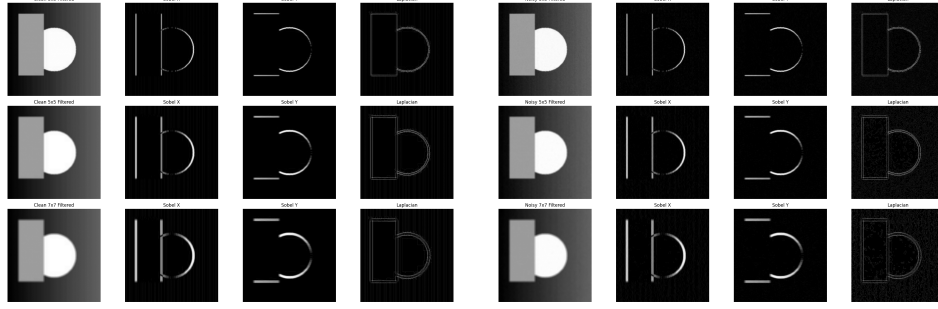


Figure 3: Q3 Example: Sobel and Laplacian edge maps.

- **Implementation:** Sobel and Laplacian filters are applied to grayscale images to extract edge information. Edge maps are saved for each image, highlighting boundaries and structural features.

#### Q4: Canny Edge Detection (from Scratch)

- **Theory:** The Canny edge detector is a multi-stage algorithm for robust edge detection. It includes Gaussian smoothing, gradient calculation, non-maximum suppression, and thresholding.
- **Formulas:**
  - Gaussian smoothing:  $I_{smooth} = I * G$ , where  $G$  is a Gaussian kernel.
  - Gradient:  $G_x, G_y$  as in Sobel; magnitude  $M = \sqrt{G_x^2 + G_y^2}$ ; direction  $\theta = \arctan(G_y/G_x)$ .
  - Non-maximum suppression: Retain local maxima along gradient direction.
  - Thresholding:  $E = (M > T)$ , where  $T$  is a chosen threshold.
- **Implementation:** A full Canny pipeline is implemented: Gaussian blur, gradient computation, non-maximum suppression, and thresholding. Final edge maps are visualized and saved, providing robust edge detection results for a variety of image types.

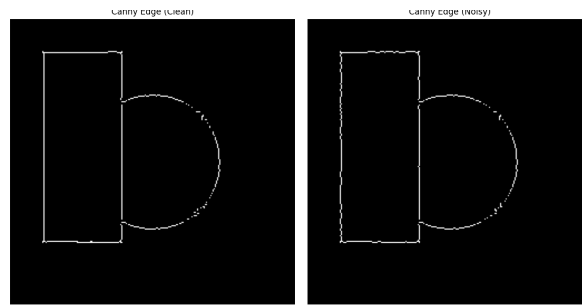


Figure 4: Q4 Example: Canny edge map.

## Results and Observations

- The scripts efficiently process a large, diverse dataset using random sampling, making the workflow scalable and reproducible.
- Output plots and images are organized by task and class, facilitating easy review and comparison.

- The modular codebase allows for rapid experimentation and extension to new tasks or datasets.
- All code avoids OpenCV and PIL, ensuring compatibility with Python 3.12 and broad platform support.
- The project demonstrates key computer vision concepts: color analysis, noise modeling, edge detection, and algorithmic implementation.

## How to Use

- Activate the Python virtual environment and install dependencies (`numpy`, `matplotlib`, `scipy`, `requests`).
- Run each script in `src/` to generate outputs for the sampled images.
- Review the generated plots in `images/plots/` for visual analysis and reporting.

## References

- Gonzalez, R. C., Woods, R. E. (2018). Digital Image Processing. Pearson.
- Scipy, Numpy, Matplotlib documentation.